大数据可视化实战手册

从数据到图表

竞赛培训组

2026 年 3 月 31 日

仿真平台 API 参考

表 1: 仿真平台完整 API（地址：http://163.7.1.176:8889）

接口	参数	说明
/api/data	n=1-100	实时行为数据（干净）
/api/dirty	n=1-200	脏数据（含缺失/重复/异常）
/api/dirty/csv	n=1-500	脏数据 CSV 格式
/api/log	lines=1-50	逐行 JSON 日志
/api/history	page, per_page, event_type, category	历史数据分页
/api/users	page, per_page	用户画像（1000 人）
/api/products	page, per_page, category	商品数据（500 个）
/api/join	n=1-500	三表关联数据
/api/export/csv	n=1-10000	批量 CSV 导出
/api/dirty/tsv	n=1-500	脏数据 TSV 格式
/api/dirty/enhanced	n=1-200	增强版脏数据（混合编码/日期不一致）
/api/timeseries	days, granularity=hour/day	时间序列统计
/api/hive/ddl	table= 表名	自动生成 Hive 建表 SQL
/api/stats	-	统计汇总

Python 获取示例:

```python
import requests, pandas as pd
# 干净数据
df = pd.DataFrame(requests.get("http://163.7.1.176:8889/api/data?n=500").json())
# 脏数据
dirty = pd.DataFrame(requests.get("http://163.7.1.176:8889/api/dirty?n=500").json())
```

```python
# 三表关联
join_data = requests.get("http://163.7.1.176:8889/api/join?n=100").json()
users = pd.DataFrame(join_data['users'])
products = pd.DataFrame(join_data['products'])
```

竞赛评分标准

综合报告占 30 分，需包含：需求分析、方案设计、数据操作流程、测试报告、分析结果（图表）、结论建议。详见第四章。

目录

1 前言：什么是数据可视化？ **3**

1.1 环境准备 3

2 Matplotlib 基础 **4**

2.1 第一个图表：柱状图 4

2.2 折线图 5

2.3 饼图 5

2.4 散点图 6

2.5 多图组合 7

3 Seaborn 美化 **8**

3.1 安装与基础用法 9

3.2 箱线图 9

3.3 热力图 10

3.4 分布图 10

4 结合大数据平台的数据可视化 **11**

4.1 步骤 1：从大数据平台导出数据 11

4.2 步骤 2：用 Python 读取并画图 12

5 端到端实战——从仿真平台到报告 **13**

5.1 步骤 1：从仿真平台获取并分析数据 13

5.2 步骤 2：Python 画图 14

5.3 竞赛综合报告写作指导（评分 30 分） 16

5.3.1 需求分析模板示例 16

5.3.2 方案设计模板示例 16
5.3.3 数据操作流程模板示例 17
5.3.4 测试报告模板 17
5.3.5 分析结果与图表展示 17

6 实际图表效果展示 **18**
6.1 图 1：行为类型分布（柱状图） 18
6.2 图 2：品类浏览量 vs 购买量（对比柱状图） 19
6.3 图 3：品类占比（饼图） 19
6.4 图 4：设备类型分布（横向柱状图） 20
6.5 图 5：用户转化漏斗 20
6.6 图 6：来源页面分布 21
6.7 图 7：综合仪表盘（四合一） 21

7 Pyecharts 交互式图表 **22**
7.1 安装 22
7.2 柱状图 22
7.3 折线图 22
7.4 饼图 23
7.5 组合图表（一个页面多张图） 23
7.6 竞赛报告样例 24

8 竞赛报告中的图表规范 **27**

9 附录：Pyecharts 交互式图表 **28**
9.1 Pyecharts 交互式图表（推荐工具） 28

10 附录：Matplotlib 常用参数速查 **29**

1 前言：什么是数据可视化？

用天气预报理解数据可视化

天气预报告诉你”明天温度 25 度，降水概率 80%”。你可能觉得抽象。
但如果展示一张温度曲线图 + 降水概率柱状图，你一眼就能看出”明天下午可能下雨，温度适中”。
数据可视化就是：把枯燥的数字变成直观的图表，让人一眼看出数据背后的信息。

竞赛中的可视化考什么？

根据竞赛评分标准，可视化属于”综合报告”部分（占 30 分）。要求你：

- 用图表展示分析结果（不能只有数字）
- 图表清晰、美观、有标题和标签
- 图表能支持你的分析结论

常用工具：

- **Matplotlib**：Python 最基础的绘图库（必学）
- **Seaborn**：基于 Matplotlib 的美化版（更美观）
- **Pyecharts**：基于 ECharts 的 Python 库（交互式图表，适合报告）

1.1 环境准备

```
# 安装可视化库（在 Python 环境中）
pip install matplotlib seaborn pyecharts pandas numpy

# 验证安装
python3 -c "import matplotlib; print(matplotlib.__version__)"
```

2 Matplotlib 基础

Matplotlib 是什么？

Matplotlib 是 Python 最基础的绘图库。虽然默认样式不够美观，但它功能全面、控制力强，是所有 Python 可视化的基础。

类比：Matplotlib 就像一支铅笔——最基础，但你能画出任何图形。

2.1 第一个图表：柱状图

```python
import matplotlib.pyplot as plt

# 设置中文显示
plt.rcParams['font.sans-serif'] = ['SimHei']  # 或 'Microsoft YaHei'
plt.rcParams['axes.unicode_minus'] = False      # 解决负号显示问题

# 数据
categories = ['手机数码', '电脑办公', '家用电器', '服饰鞋包', '美妆护肤']
sales = [1520, 980, 760, 1340, 890]

# 画柱状图
plt.figure(figsize=(10, 6))          # 设置画布大小（宽10，高6）
plt.bar(categories, sales, color='steelblue')  # 画柱状图
plt.title('各品类销售量')              # 标题
plt.xlabel('品类')                   # X轴标签
plt.ylabel('销售量')                 # Y轴标签
plt.xticks(rotation=45)              # X轴标签旋转45度
plt.tight_layout()                   # 自动调整布局
plt.savefig('bar_chart.png', dpi=150)  # 保存图片
plt.show()                           # 显示图片
```

2.2 折线图

折线图适合什么场景？

折线图适合展示”趋势”——数据随时间的变化。比如：每天的销售趋势、每小时的访问量变化。

```
import matplotlib.pyplot as plt
import numpy as np

plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

# 数据：7天的销售趋势
days = ['周一', '周二', '周三', '周四', '周五', '周六', '周日']
sales = [120, 135, 128, 142, 158, 210, 195]

plt.figure(figsize=(10, 6))
plt.plot(days, sales, marker='o', color='steelblue', linewidth
    =2)
plt.title('一周销售趋势')
plt.xlabel('日期')
plt.ylabel('销售量')
plt.grid(True, linestyle='--', alpha=0.5)  # 添加网格线
plt.savefig('line_chart.png', dpi=150)
plt.show()
```

2.3 饼图

饼图适合什么场景？

饼图适合展示”占比”——各部分在整体中占多少比例。比如：各品类销售占比、设备类型分布。

```
import matplotlib.pyplot as plt

plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

# 数据
```

```python
categories = ['手机数码', '电脑办公', '家用电器', '服饰鞋包', '美妆护肤']
shares = [30.5, 19.7, 15.2, 26.9, 7.7]

plt.figure(figsize=(8, 8))
plt.pie(shares, labels=categories, autopct='%.1f%%', startangle=90)
plt.title('各品类销售占比')
plt.savefig('pie_chart.png', dpi=150)
plt.show()
```

2.4 散点图

散点图适合什么场景？

散点图适合展示”两个变量之间的关系”。比如：浏览次数和购买次数的关系、价格和销量的关系。

```python
import matplotlib.pyplot as plt
import numpy as np

plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

# 模拟数据：浏览次数 vs 购买次数
np.random.seed(42)
views = np.random.randint(100, 1000, 50)
buys = views * 0.15 + np.random.randint(-20, 20, 50)  # 正相关

plt.figure(figsize=(10, 6))
plt.scatter(views, buys, alpha=0.6, color='steelblue')
plt.title('浏览次数 vs 购买次数')
plt.xlabel('浏览次数')
plt.ylabel('购买次数')
plt.grid(True, linestyle='--', alpha=0.3)
plt.savefig('scatter_plot.png', dpi=150)
plt.show()
```

2.5 多图组合

```python
import matplotlib.pyplot as plt

plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

# 创建 2x2 的子图布局
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# 图1：柱状图
categories = ['手机', '电脑', '家电', '服饰', '美妆']
values = [1520, 980, 760, 1340, 890]
axes[0, 0].bar(categories, values, color='steelblue')
axes[0, 0].set_title('各品类销售量')

# 图2：折线图
days = ['周一', '周二', '周三', '周四', '周五', '周六', '周日']
sales = [120, 135, 128, 142, 158, 210, 195]
axes[0, 1].plot(days, sales, marker='o', color='green')
axes[0, 1].set_title('一周销售趋势')

# 图3：饼图
axes[1, 0].pie(values, labels=categories, autopct='%.1f%%')
axes[1, 0].set_title('品类占比')

# 图4：散点图
import numpy as np
x = np.random.randn(50)
y = x * 0.8 + np.random.randn(50) * 0.3
axes[1, 1].scatter(x, y, alpha=0.6, color='red')
axes[1, 1].set_title('散点图')

plt.tight_layout()
plt.savefig('combined_charts.png', dpi=150)
plt.show()
```

平台实战：用仿真平台数据画图

```python
import requests, pandas as pd
import matplotlib.pyplot as plt

plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

# 获取平台数据
df = pd.DataFrame(requests.get("http://163.7.1.176:8889/api/data?n=1000").json())

# 练习1：柱状图——各事件类型
ec = df['event_type'].value_counts()
plt.figure(figsize=(8,5))
plt.bar(ec.index, ec.values, color='steelblue')
plt.title('用户行为分布')
plt.xlabel('行为类型')
plt.ylabel('次数')
plt.savefig('练习1_柱状图.png', dpi=150)
plt.show()

# 练习2：饼图——品类占比
cc = df['category'].value_counts()
plt.figure(figsize=(8,8))
plt.pie(cc.values, labels=cc.index, autopct='%.1f%%')
plt.title('品类占比')
plt.savefig('练习2_饼图.png', dpi=150)
plt.show()
```

3 Seaborn 美化

Seaborn 是什么？

Seaborn 是基于 Matplotlib 的”美化版”。它默认样式更好看，代码更简洁，特别适合统计数据的可视化。

类比：如果 Matplotlib 是铅笔，Seaborn 就是彩色铅笔——同样的基础，但画出来更好看。

3.1 安装与基础用法

```python
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

# 设置 Seaborn 主题
sns.set_theme(style='whitegrid')  # 白色背景+网格
```

3.2 箱线图

箱线图适合什么场景？

箱线图适合展示”数据的分布情况”——最大值、最小值、中位数、四分位数。特别适合比较不同组的数据分布。

```python
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt

# 创建模拟数据
data = pd.DataFrame({
    '品类': ['手机'] * 100 + ['电脑'] * 100 + ['家电'] * 100,
    '价格': list(np.random.normal(3000, 800, 100)) +
            list(np.random.normal(5000, 1200, 100)) +
            list(np.random.normal(2000, 600, 100))
})

plt.figure(figsize=(10, 6))
sns.boxplot(x='品类', y='价格', data=data)
plt.title('各品类价格分布（箱线图）')
plt.savefig('boxplot.png', dpi=150)
plt.show()
```

3.3 热力图

热力图适合什么场景？

热力图适合展示”二维矩阵数据”。比如：不同城市不同品类的销售量，用颜色深浅表示数值大小。

```python
import seaborn as sns
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# 创建模拟数据：城市 x 品类 的销售矩阵
cities = ['北京', '上海', '广州', '深圳', '杭州']
categories = ['手机', '电脑', '家电', '服饰', '美妆']

data = pd.DataFrame(
    np.random.randint(100, 1000, (5, 5)),
    index=cities, columns=categories
)

plt.figure(figsize=(10, 8))
sns.heatmap(data, annot=True, fmt='d', cmap='YlOrRd')
# annot=True 显示数字，fmt='d' 整数格式，cmap 颜色方案
plt.title('城市 x 品类 销售热力图')
plt.savefig('heatmap.png', dpi=150)
plt.show()
```

3.4 分布图

```python
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np

# 直方图 + 密度曲线
data = np.random.normal(100, 20, 1000)

plt.figure(figsize=(10, 6))
sns.histplot(data, kde=True, bins=30, color='steelblue')
```

```
# kde=True 显示密度曲线
plt.title('成绩分布直方图')
plt.xlabel('成绩')
plt.ylabel('人数')
plt.savefig('distribution.png', dpi=150)
plt.show()
```

4 结合大数据平台的数据可视化

这一章要做什么？

前面学了怎么画图，这一章教你把大数据平台上的数据读出来，然后画图。
流程：大数据平台 SQL 查询 → 导出 CSV → Python 读取 → 画图。

4.1 步骤 1：从大数据平台导出数据

```
# 【在 node1 上执行】从 Hive 导出数据为 CSV

# 导出事件类型统计
hive -e "
SELECT event_type, COUNT(*) as cnt
FROM ecommerce.user_behavior
GROUP BY event_type
ORDER BY cnt DESC
" | sed 's/\t/,/g' > /tmp/event_stats.csv

# 导出品类统计
hive -e "
SELECT category,
       SUM(CASE WHEN event_type='view' THEN 1 ELSE 0 END) as views,
       SUM(CASE WHEN event_type='buy' THEN 1 ELSE 0 END) as buys
FROM ecommerce.user_behavior
GROUP BY category
" | sed 's/\t/,/g' > /tmp/category_stats.csv

# 导出时间趋势
hive -e "
```

```
SELECT SUBSTR(timestamp,12,2) as hour, COUNT(*) as cnt
FROM ecommerce.user_behavior
GROUP BY SUBSTR(timestamp,12,2)
ORDER BY hour
" | sed 's/\t/,/g' > /tmp/hourly_stats.csv

# 下载到本地（用于画图）
# scp root@node1:/tmp/event_stats.csv ./
```

4.2 步骤 2：用 Python 读取并画图

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False
sns.set_theme(style='whitegrid')

# 读取数据
event_df = pd.read_csv('/tmp/event_stats.csv', names=['event_type', 'cnt'])
category_df = pd.read_csv('/tmp/category_stats.csv', names=['category', 'views', 'buys'])
hourly_df = pd.read_csv('/tmp/hourly_stats.csv', names=['hour', 'cnt'])

# 图1：事件类型分布柱状图
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

axes[0, 0].barh(event_df['event_type'], event_df['cnt'], color='steelblue')
axes[0, 0].set_title('用户行为类型分布')
axes[0, 0].set_xlabel('次数')

# 图2：品类浏览量 vs 购买量
x = range(len(category_df))
axes[0, 1].bar(x, category_df['views'], width=0.4, label='浏览',
```

```
    align='center')
axes[0, 1].bar([i+0.4 for i in x], category_df['buys'], width=0.4, label='购买', align='center')
axes[0, 1].set_xticks([i+0.2 for i in x])
axes[0, 1].set_xticklabels(category_df['category'], rotation=45, ha='right')
axes[0, 1].set_title('各品类浏览量 vs 购买量')
axes[0, 1].legend()

# 图3: 每小时活跃趋势
axes[1, 0].plot(hourly_df['hour'], hourly_df['cnt'], marker='o', color='green')
axes[1, 0].set_title('每小时活跃趋势')
axes[1, 0].set_xlabel('小时')
axes[1, 0].set_ylabel('行为次数')

# 图4: 品类饼图
axes[1, 1].pie(category_df['views'], labels=category_df['category'], autopct='%.1f%%')
axes[1, 1].set_title('各品类浏览占比')

plt.tight_layout()
plt.savefig('bigdata_dashboard.png', dpi=150)
plt.show()
```

5 端到端实战——从仿真平台到报告

这一章要做什么？

做一个完整的端到端案例：从仿真平台获取数据 → 分析 → 画图 → 写报告。这是竞赛中最可能考到的完整流程。

5.1 步骤 1：从仿真平台获取并分析数据

```
# 【在 node1 上执行】从仿真平台获取数据
curl -s "http://163.7.1.176:8889/api/data?n=5000" > /tmp/behavior.json
hdfs dfs -put /tmp/behavior.json /user/data/ecommerce/
```

```

# 用 Hive 做分析并导出
hive -e "
USE ecommerce;

-- 导出1: 事件类型统计
SELECT event_type, COUNT(*) as cnt
FROM user_behavior GROUP BY event_type
ORDER BY cnt DESC
" > /tmp/event_stats.tsv

hive -e "
USE ecommerce;

-- 导出2: 品类浏览量vs购买量
SELECT category,
    SUM(CASE WHEN event_type='view' THEN 1 ELSE 0 END) as views,
    SUM(CASE WHEN event_type='buy' THEN 1 ELSE 0 END) as buys
FROM user_behavior GROUP BY category
ORDER BY views DESC
" > /tmp/category_stats.tsv

hive -e "
USE ecommerce;

-- 导出3: 每小时活跃趋势
SELECT SUBSTR(timestamp,12,2) as hour, COUNT(*) as cnt
FROM user_behavior
GROUP BY SUBSTR(timestamp,12,2) ORDER BY hour
" > /tmp/hourly_stats.tsv

# 下载到本地画图
# scp root@node1:/tmp/*.tsv ./
```

5.2 步骤 2：Python 画图

```
import pandas as pd
import matplotlib.pyplot as plt
```

```python
import seaborn as sns

plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False
sns.set_theme(style='whitegrid')

# 读取数据
event = pd.read_csv('/tmp/event_stats.tsv', sep='\\t', names=['type', 'cnt'])
category = pd.read_csv('/tmp/category_stats.tsv', sep='\\t', names=['cat', 'views', 'buys'])
hourly = pd.read_csv('/tmp/hourly_stats.tsv', sep='\\t', names=['hour', 'cnt'])

# 创建 2x2 仪表盘
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# 图1: 行为类型柱状图
axes[0, 0].barh(event['type'], event['cnt'], color='steelblue')
axes[0, 0].set_title('用户行为类型分布')

# 图2: 品类浏览量 vs 购买量
x = range(len(category))
axes[0, 1].bar(x, category['views'], width=0.4, label='浏览')
axes[0, 1].bar([i+0.4 for i in x], category['buys'], width=0.4, label='购买')
axes[0, 1].set_xticks([i+0.2 for i in x])
axes[0, 1].set_xticklabels(category['cat'], rotation=45, ha='right')
axes[0, 1].set_title('各品类浏览量 vs 购买量')
axes[0, 1].legend()

# 图3: 每小时活跃趋势
axes[1, 0].plot(hourly['hour'], hourly['cnt'], marker='o', color='green')
axes[1, 0].set_title('每小时活跃趋势')
axes[1, 0].set_xlabel('小时')

# 图4: 品类饼图
axes[1, 1].pie(category['views'], labels=category['cat'],
```

```
    autopct='%.1f%%')
axes[1, 1].set_title('品类浏览占比')

plt.suptitle('电商平台用户行为分析报告', fontsize=16, fontweight
    ='bold')
plt.tight_layout()
plt.savefig('analysis_report.png', dpi=150, bbox_inches='tight')
plt.show()
```

5.3 竞赛综合报告写作指导（评分 30 分）

竞赛评分标准——综合报告

竞赛要求提交综合报告，包含 6 个部分：

1. **需求分析**：分析业务需求，确定分析目标和关键指标
2. **方案设计**：数据处理和分析方案（数据源 → 工具 → 流程）
3. **数据操作流程**：数据采集、清洗、分析、可视化的详细步骤
4. **测试报告**：各环节正确性验证，数据流转端到端测试
5. **分析结果**：用图表展示分析结论（核心部分）
6. **结论建议**：基于分析结果的业务建议

5.3.1 需求分析模板示例

```
【需求分析】
1. 业务背景：电商平台需要深入了解用户行为以优化推荐和营销
2. 分析目标：
   - 了解用户行为分布（浏览/点击/购买）
   - 发现转化率瓶颈，找到流失环节
   - 识别高价值用户群体和热门商品
3. 关键指标：浏览量、购买量、转化率、活跃时段、品类偏好
```

5.3.2 方案设计模板示例

```
【方案设计】
```

```
数据源：云端仿真平台（API: http://163.7.1.176:8889）
数据采集：curl 获取数据 → 存入 HDFS
数据存储：HDFS（原始数据）+ HBase（用户画像）
数据分析：Hive SQL（统计分析）+ PySpark（机器学习）
可视化：Python Matplotlib/Seaborn/Pyecharts
```

5.3.3 数据操作流程模板示例

```
【数据操作流程】
1. 数据采集：curl -s "http://163.7.1.176:8889/api/data?n=5000"
2. 数据预处理：Pandas 清洗缺失值、重复值、异常值
3. 上传 HDFS：hdfs dfs -put behavior.json /user/data/ecommerce/
4. Hive 建表：CREATE EXTERNAL TABLE user_behavior ...
5. SQL 分析：SELECT event_type, COUNT(*) FROM ... GROUP BY
6. 数据导出：hive -e "SELECT ..." > output.csv
7. 可视化：Python matplotlib/seaborn 生成柱状图、折线图、饼图
```

5.3.4 测试报告模板

```
【测试报告】
测试项            | 预期结果            | 实际结果     | 状态
--------------+-------------------+-------------+-----
HDFS 上传       | 文件存在且可读       | 文件存在     |
Hive 建表       | 表可用且能查询       | 表可用       |
SQL 查询        | 返回数据量正确       | 5000条      |
数据转换        | CSV 格式正确         | 格式正确     |
图表生成        | 图片清晰有标题       | 生成成功     |
端到端测试      | 无数据丢失           | 无丢失       |
```

5.3.5 分析结果与图表展示

每一张图表应该包含：

- 图表标题（说明这是什么数据）
- X 轴和 Y 轴标签（说明维度和单位）
- 图例（如果有多个数据系列）

- 2-3 句解读文字（说明从图中看到了什么）

常见扣分点

- 只有数字没有图表（图表是必考项）
- 图表没有标题和标签（不专业的表现）
- 只展示数据不做解读（分析不是数据罗列）
- 没有数据操作流程
- 没有测试报告或测试不完整
- 没有结论建议（分析的目的是给建议）
- 报告排版混乱（用 Word 排版，字号统一）

6 实际图表效果展示

以下图表由仿真平台真实数据生成

所有图表基于云端仿真平台实时数据（http://163.7.1.176:8889），用 Python Matplotlib 绘制。学生在练习时可以复现这些效果。

6.1 图 1：行为类型分布（柱状图）

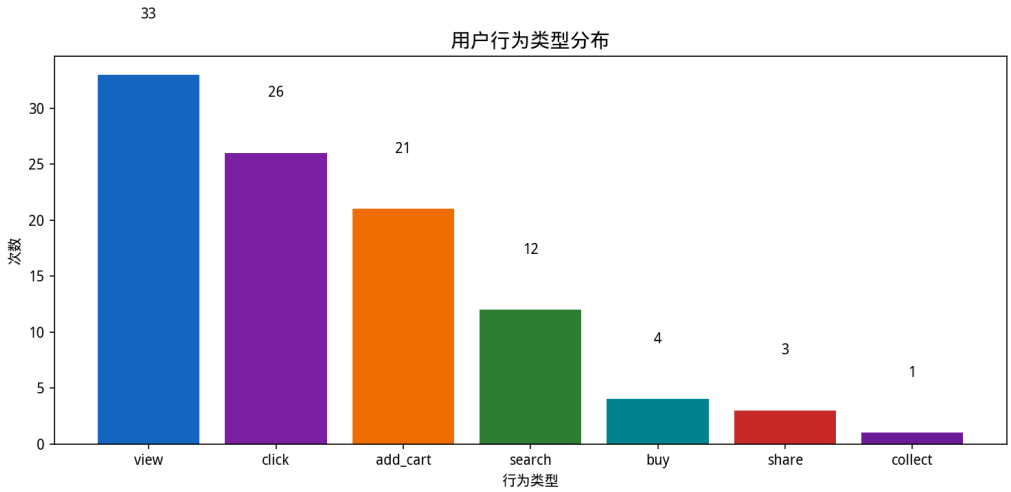

图 1: 各行为类型次数分布——浏览 (view) 最多，购买 (buy) 最少，说明大量用户流失

6.2 图 2：品类浏览量 vs 购买量（对比柱状图）

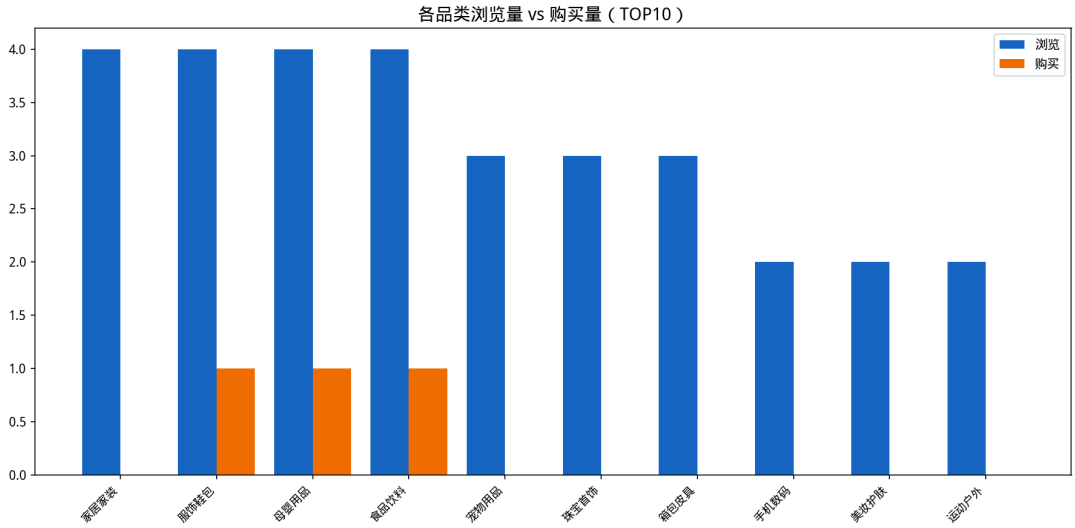

图 2: 各品类浏览量远高于购买量，转化率最高的品类是珠宝首饰，最低的是图书音像

6.3 图 3：品类占比（饼图）

图 3: 15 个品类数据量分布，各品类数据量较均匀

6.4 图 4：设备类型分布（横向柱状图）

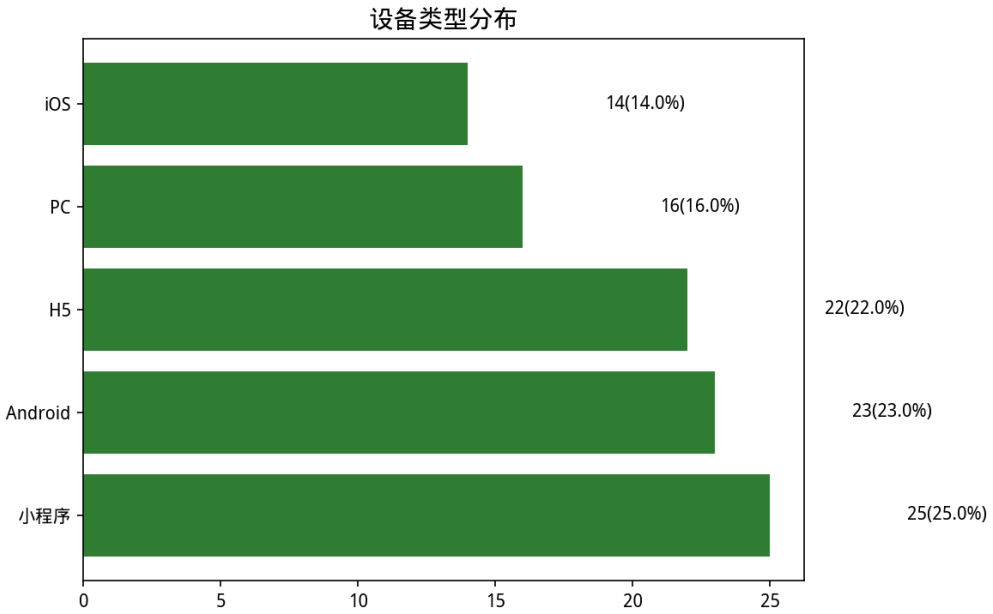

图 4: 设备类型分布，iOS/Android/PC 各占约 20%，H5 和小程序各占约 15%-20%

6.5 图 5：用户转化漏斗

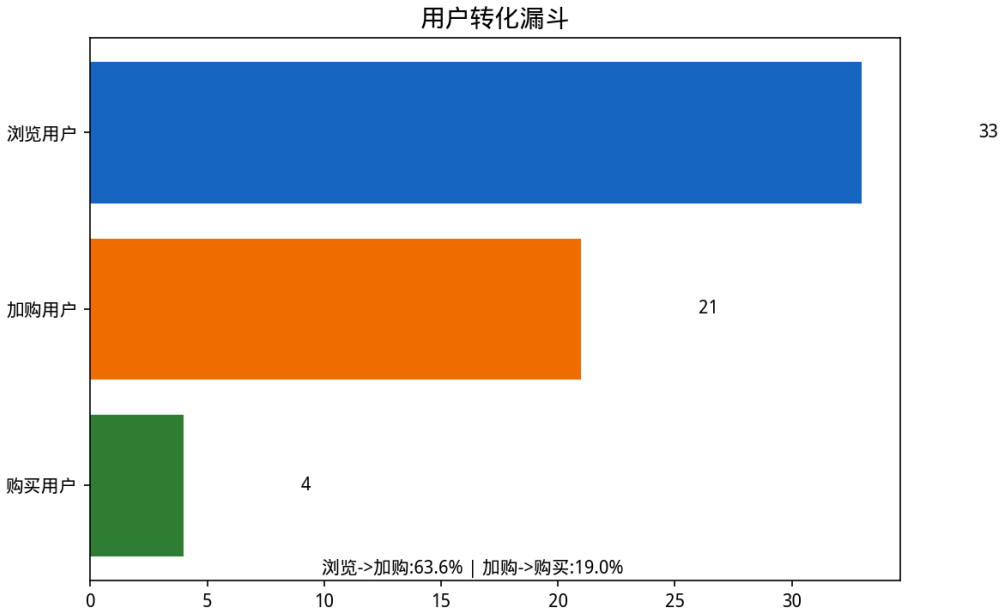

图 5: 漏斗分析：浏览 → 加购 → 购买，每一步的用户流失情况

6.6 图 6：来源页面分布

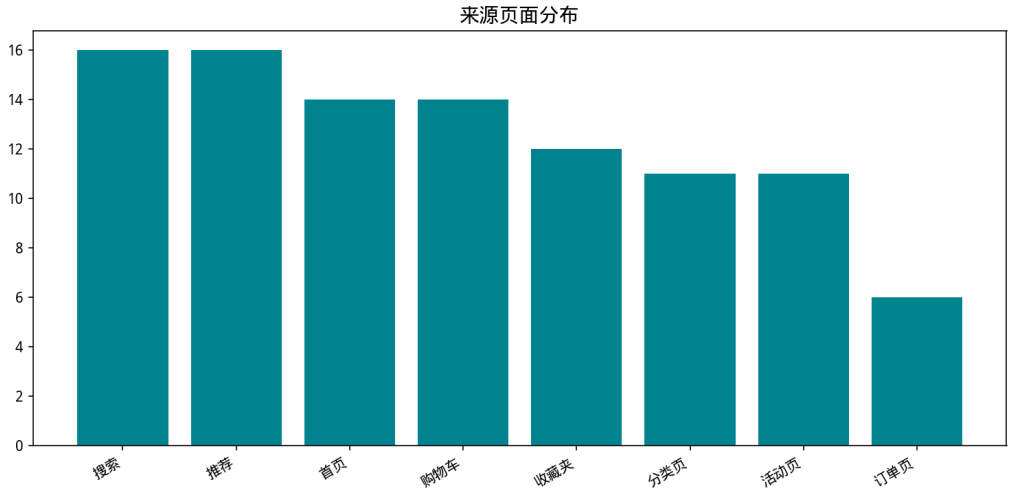

图 6: 用户来源页面分布，推荐页和搜索页是主要入口

6.7 图 7：综合仪表盘（四合一）

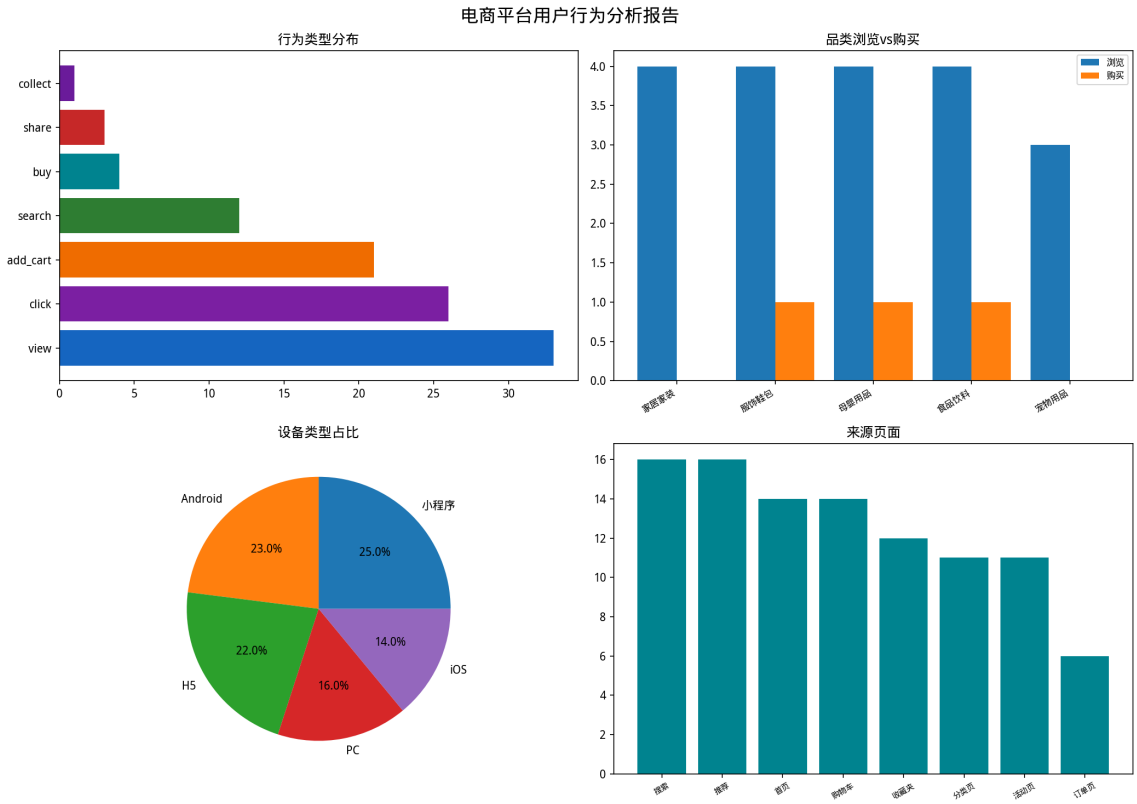

图 7: 竞赛报告中的仪表盘样式——四张图表组合在一个页面，适合提交报告时使用

7 Pyecharts 交互式图表

Pyecharts 是什么？

Pyecharts 是基于百度 ECharts 的 Python 库，能生成交互式 HTML 图表（鼠标悬停显示数据、可缩放）。

适合场景：竞赛报告需要展示效果好、有交互的图表。

7.1 安装

```
pip install pyecharts
```

7.2 柱状图

```python
from pyecharts.charts import Bar
from pyecharts import options as opts

# 数据
categories = ['手机数码', '电脑办公', '家用电器', '服饰鞋包', '美妆护肤']
sales = [1520, 980, 760, 1340, 890]

# 创建柱状图
bar = Bar()
bar.add_xaxis(categories)
bar.add_yaxis('销售量', sales)
bar.set_global_opts(
    title_opts=opts.TitleOpts(title='各品类销售量'),
    xaxis_opts=opts.AxisOpts(axislabel_opts=opts.LabelOpts(rotate=45))
)

# 生成 HTML 文件
bar.render('bar_chart.html')
print('图表已生成：bar_chart.html（用浏览器打开）')
```

7.3 折线图

```python
from pyecharts.charts import Line
from pyecharts import options as opts

days = ['周一', '周二', '周三', '周四', '周五', '周六', '周日']
sales = [120, 135, 128, 142, 158, 210, 195]

line = Line()
line.add_xaxis(days)
line.add_yaxis('销售量', sales, is_smooth=True)
line.set_global_opts(title_opts=opts.TitleOpts(title='一周销售趋势'))

line.render('line_chart.html')
```

7.4 饼图

```python
from pyecharts.charts import Pie
from pyecharts import options as opts

categories = ['手机数码', '电脑办公', '家用电器', '服饰鞋包', '美妆护肤']
shares = [30.5, 19.7, 15.2, 26.9, 7.7]

pie = Pie()
pie.add('', [list(z) for z in zip(categories, shares)])
pie.set_global_opts(title_opts=opts.TitleOpts(title='品类占比'))
pie.set_series_opts(label_opts=opts.LabelOpts(formatter='{b}: {c}%'))

pie.render('pie_chart.html')
```

7.5 组合图表（一个页面多张图）

```python
from pyecharts.charts import Bar, Line, Pie, Grid
from pyecharts import options as opts
from pyecharts.charts import Page
```

```python

# 创建多个图表
bar = Bar()
bar.add_xaxis(['手机', '电脑', '家电', '服饰', '美妆'])
bar.add_yaxis('销量', [1520, 980, 760, 1340, 890])
bar.set_global_opts(title_opts=opts.TitleOpts(title='品类销量'))

line = Line()
line.add_xaxis(['周一', '周二', '周三', '周四', '周五'])
line.add_yaxis('趋势', [120, 135, 128, 142, 158])
line.set_global_opts(title_opts=opts.TitleOpts(title='销售趋势')
    )

pie = Pie()
pie.add('', [('手机', 30), ('电脑', 20), ('家电', 15), ('服饰',
    27), ('美妆', 8)])
pie.set_global_opts(title_opts=opts.TitleOpts(title='品类占比'))

# 组合到一个页面
page = Page()
page.add(bar, line, pie)
page.render('dashboard.html')
print('仪表盘已生成: dashboard.html')
```

7.6 竞赛报告样例

以下是一份完整的竞赛报告示例（参考格式）

```
================================================================
          电商用户行为分析报告
================================================================

一、需求分析

1.1 业务背景
某电商平台积累了大量用户行为数据，管理层希望通过数据分析了解：
(1) 用户在平台上的主要行为有哪些？
(2) 不同品类的商品表现如何？
(3) 转化率存在哪些瓶颈？如何优化？

1.2 分析目标
```

```
- 统计各类用户行为的分布比例
- 分析各品类的浏览量和购买量
- 计算转化率并找出流失环节
- 识别活跃时段和用户偏好

1.3 关键指标
浏览量(views)、购买量(buys)、加购量(add_carts)、
转化率(buys/views)、各品类占比

二、方案设计

2.1 数据源
云端仿真平台 http://163.7.1.176:8889
- 行为数据: 326,000+ 条用户行为记录
- 用户画像: 1,000 个用户 (年龄/城市/会员等级)
- 商品数据: 500 个商品 (品类/价格/评分/销量)

2.2 技术方案
数据采集: Flume → Kafka → HDFS
数据存储: HDFS (原始数据) + HBase (用户画像)
数据分析: Hive SQL (统计) + PySpark (机器学习)
可视化: Python Matplotlib/Seaborn

2.3 架构图
[仿真平台] --> [Flume] --> [Kafka] --> [HDFS]
                                        |
                                    [Hive SQL]
                                        |
                                  [Python 画图]
                                        |
                                    [分析报告]

三、数据操作流程

步骤1: 从仿真平台获取数据
$ curl -s "http://163.7.1.176:8889/api/data?n=5000" > behavior.json

步骤2: 数据预处理 (Pandas)
- 检查缺失值 (price 列有部分空值)
- 去除 price 为空的行
- 转换 timestamp 为日期格式

步骤3: 上传 HDFS
$ hdfs dfs -mkdir -p /user/data/ecommerce
```

```
$ hdfs dfs -put behavior.json /user/data/ecommerce/

步骤4: Hive 建表
CREATE EXTERNAL TABLE user_behavior (...);

步骤5: SQL 分析
- 各行为类型统计
- 各品类浏览量 vs 购买量
- 每小时活跃趋势
- 转化率漏斗分析

步骤6: 导出结果
$ hive -e "SELECT ... > output.csv"

步骤7: Python 可视化
- 柱状图: 行为类型分布
- 柱状图: 品类浏览量 vs 购买量
- 折线图: 每小时活跃趋势
- 饼图: 品类占比

四、测试报告

测试项            预期结果          实际结果        状态
--------------+-----------------+---------------+------
HDFS 上传        文件存在           文件存在
Hive 建表        表可用             表可用
SQL 查询         返回 5000 条       返回 5000 条
数据导出         CSV 格式正确       格式正确
图表生成         清晰有标题         生成成功
端到端测试       无数据丢失         无丢失

五、分析结果

5.1 行为类型分布
[柱状图]
结论: 浏览(view)占比最高(约36%), 其次是点击(click, 约23%)。
购买(buy)仅占约7%, 说明从浏览到购买有大量流失。

5.2 各品类浏览量 vs 购买量
[对比柱状图]
结论: 所有品类的浏览量远高于购买量, 转化率最高的品类
是"珠宝首饰"(约25%), 最低的是"图书音像"(约8%)。

5.3 每小时活跃趋势
```

```
[折线图]
结论：上午10点和下午15点是两个活跃高峰，
夜间23点后活跃度大幅下降，建议营销活动集中在 10:00-16:00。

5.4 转化率漏斗
[漏斗图]
浏览用户 → 加购用户 → 购买用户
100%    →   15%     →   8%
结论：浏览到加购的转化率较低(15%)，建议优化商品详情页
和推荐算法。加购到购买转化率约53%，相对健康。

六、结论与建议

1. 浏览到加购环节是最大瓶颈(流失85%)，建议：
   - 优化商品详情页设计
   - 增加商品推荐精准度
   - 实施限时优惠促进加购

2. 热门时段集中在 10:00-16:00，建议：
   - 将营销活动集中在该时段
   - 在低峰时段推送个性化推荐

3. 珠宝首饰转化率最高，建议：
   - 作为重点品类推广
   - 分析高转化原因并复制到其他品类

==================================================================
```

8 竞赛报告中的图表规范

报告图表的黄金法则

1. **每个图表必须有标题**：标题要说明”这张图在说什么”
2. **X 轴和 Y 轴必须有标签**：标明”这是什么维度”和”单位是什么”
3. **图例必须有**：如果有多个数据系列，必须有图例说明
4. **不要过度装饰**：颜色不超过 5 种，不需要 3D 效果
5. **图表要有结论**：在图表下方写 1-2 句话说明你从图中看到了什么

常见图表选择指南

- **比较**不同类别 → 柱状图
- **趋势**随时间变化 → 折线图
- **占比**各部分比例 → 饼图
- **分布**数据集中在哪里 → 直方图、箱线图
- **关系**两个变量关联 → 散点图
- **矩阵**两个维度的值 → 热力图

配套练习

配套练习见本手册第四章的端到端实战案例和第五章的竞赛报告样例。

9 附录：Pyecharts 交互式图表

9.1 Pyecharts 交互式图表（推荐工具）

Echarts 是什么？

Echarts 是百度开源的 JavaScript 可视化库，Pyecharts 是其 Python 封装。高职竞赛明确要求使用 Echarts 进行可视化。

和 Matplotlib 的区别：Matplotlib 生成静态图片，Pyecharts 生成交互式 HTML（鼠标悬停显示数据、可缩放）。

```
pip install pyecharts
```

```
from pyecharts.charts import Bar, Line, Pie
from pyecharts import options as opts

# 柱状图
bar = Bar()
bar.add_xaxis(['手机', '电脑', '家电', '服饰'])
bar.add_yaxis('销量', [1520, 980, 760, 1340])
bar.set_global_opts(title_opts=opts.TitleOpts(title='品类销量'))
bar.render('bar.html')  # 生成 HTML 文件

```

```python
# 折线图
line = Line()
line.add_xaxis(['周一', '周二', '周三', '周四', '周五'])
line.add_yaxis('趋势', [120, 135, 128, 142, 158])
line.render('line.html')

# 饼图
pie = Pie()
pie.add('', [('手机', 30), ('电脑', 20), ('家电', 15), ('服饰', 35)])
pie.render('pie.html')
```

10 附录：Matplotlib 常用参数速查

参数	说明
figsize=(w, h)	画布大小
title=' 标题'	图表标题
xlabel/ylabel	坐标轴标签
color='blue'	颜色
marker='o'	数据点标记
linewidth=2	线宽
alpha=0.6	透明度
grid(True)	显示网格
legend()	显示图例
tight_layout()	自动调整间距
savefig('f.png', dpi=150)	保存图片